COMP2922: Models of Computation (Adv)

Lecture 5: Alternating Automata

Linus Cooper

The University of Sydney

September 1, 2023

Recap: Automata strengths/weaknesses

- From last lecture and the tutorial, we have seen what automata can and can't do easily
- DFAs: easy complement, medium union/intersect, hard reversal/concatenation/star
- NFAs: easy union/reversal/concatenation/star, medium intersect, hard complement
- NFAs seem pretty cool, but the exponential blowup on complement is annoying
- Complement is such a basic operation! Can't we get a stronger model that can handle it?
- Prepare for...

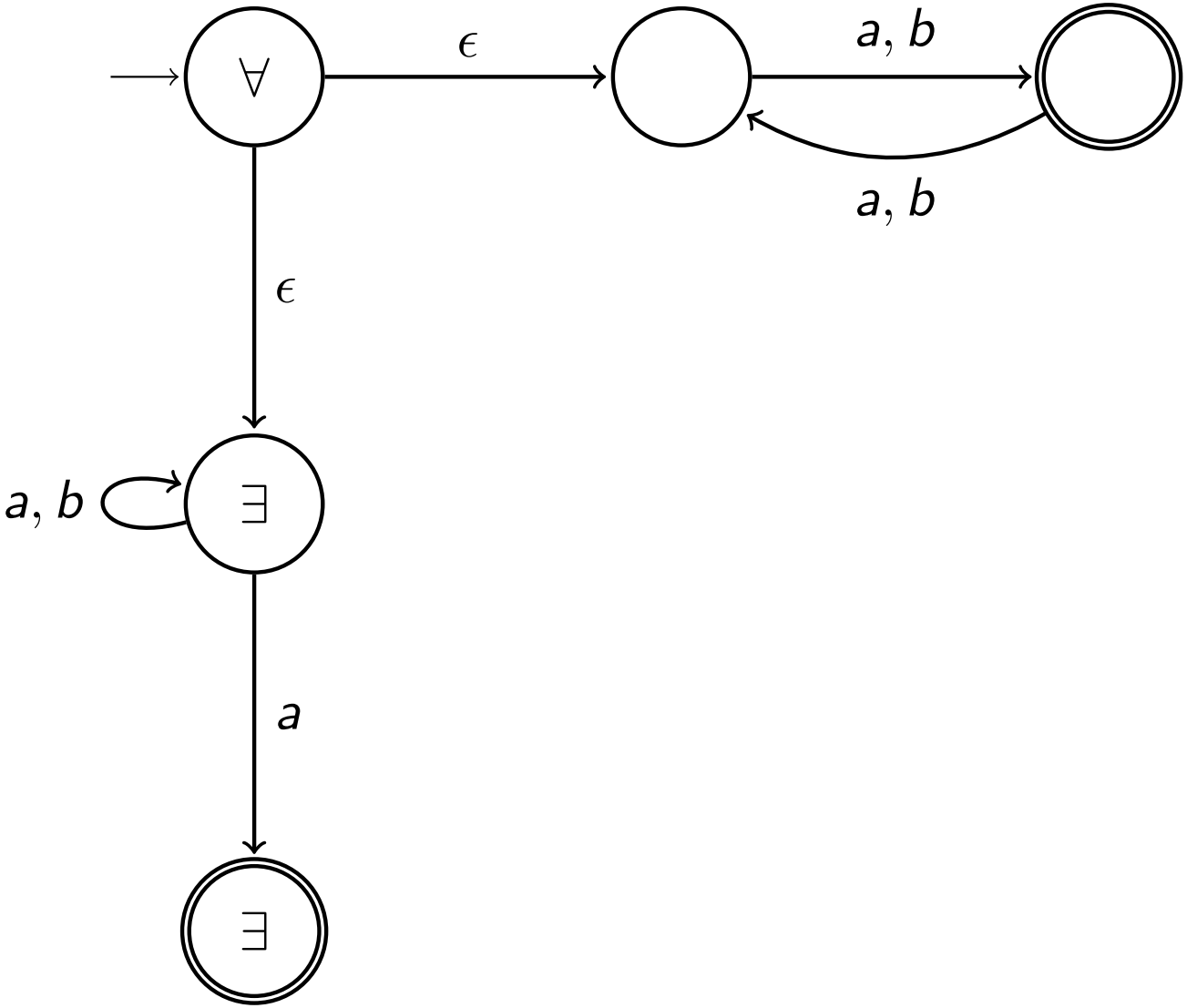
Alternating automata

An alternating finite automaton (AFA) is a tuple $(Q, \Sigma, \delta, q_0, F, T)$ where $T : Q \to \{\exists, \forall\}$ is the **type function** and the others are the same as for NFAs.

- Intuitive semantics: the existential states (q where $T(q) = \exists$) work the same way as NFAs.
- The universal states (q where $T(q) = \forall$) mean that the automaton transitions to all of the subsequent states, and accepts only if there is an accepting run from all of them.
- In diagrams, states will be labelled with the type of transition from them (or unlabelled if it doesn't matter)
- Note: For technical reasons, we disallow cycles of epsilon transitions.

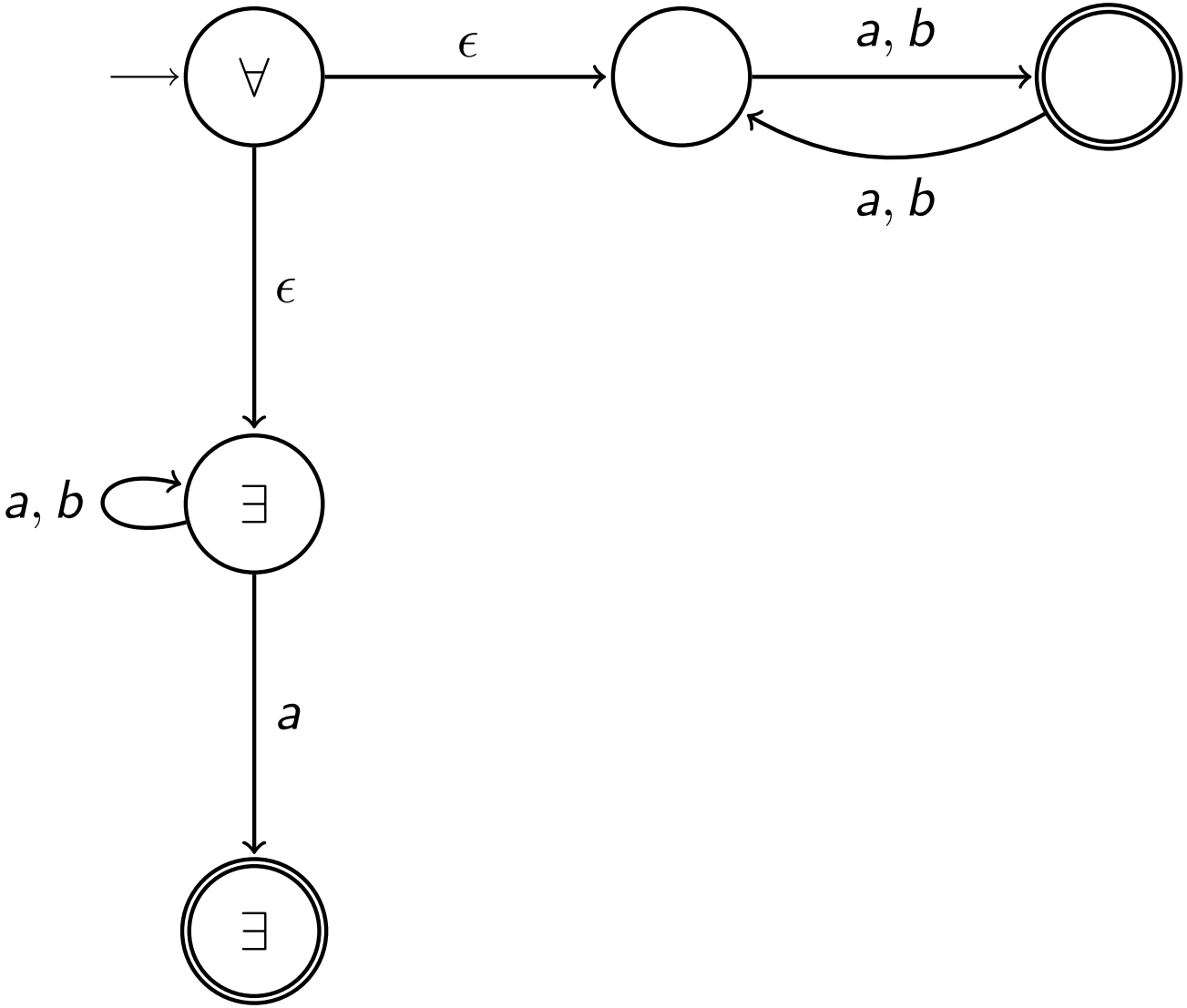
AFA 1

What does this automaton do?



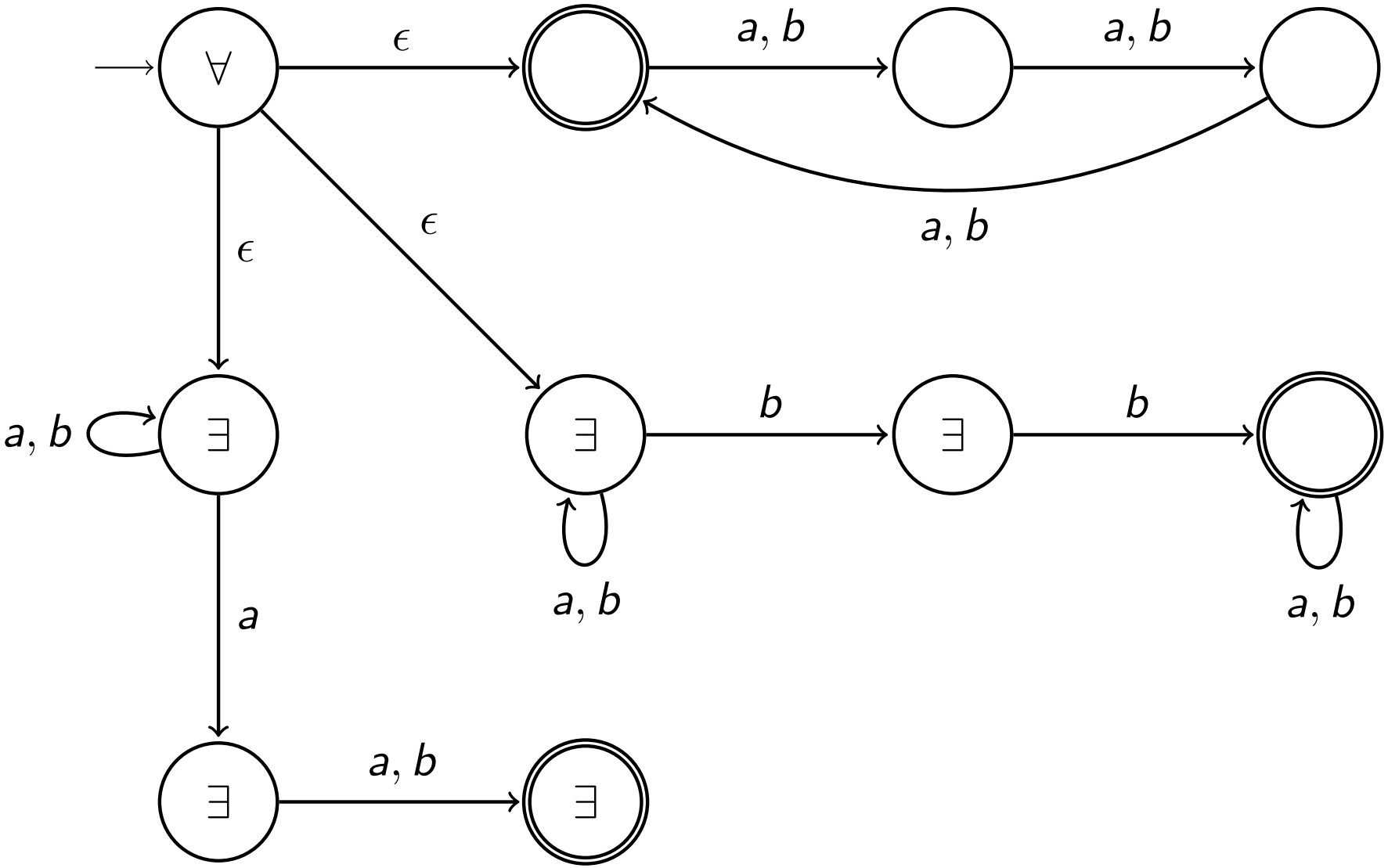
AFA 1

Length of string is odd, and string ends in a



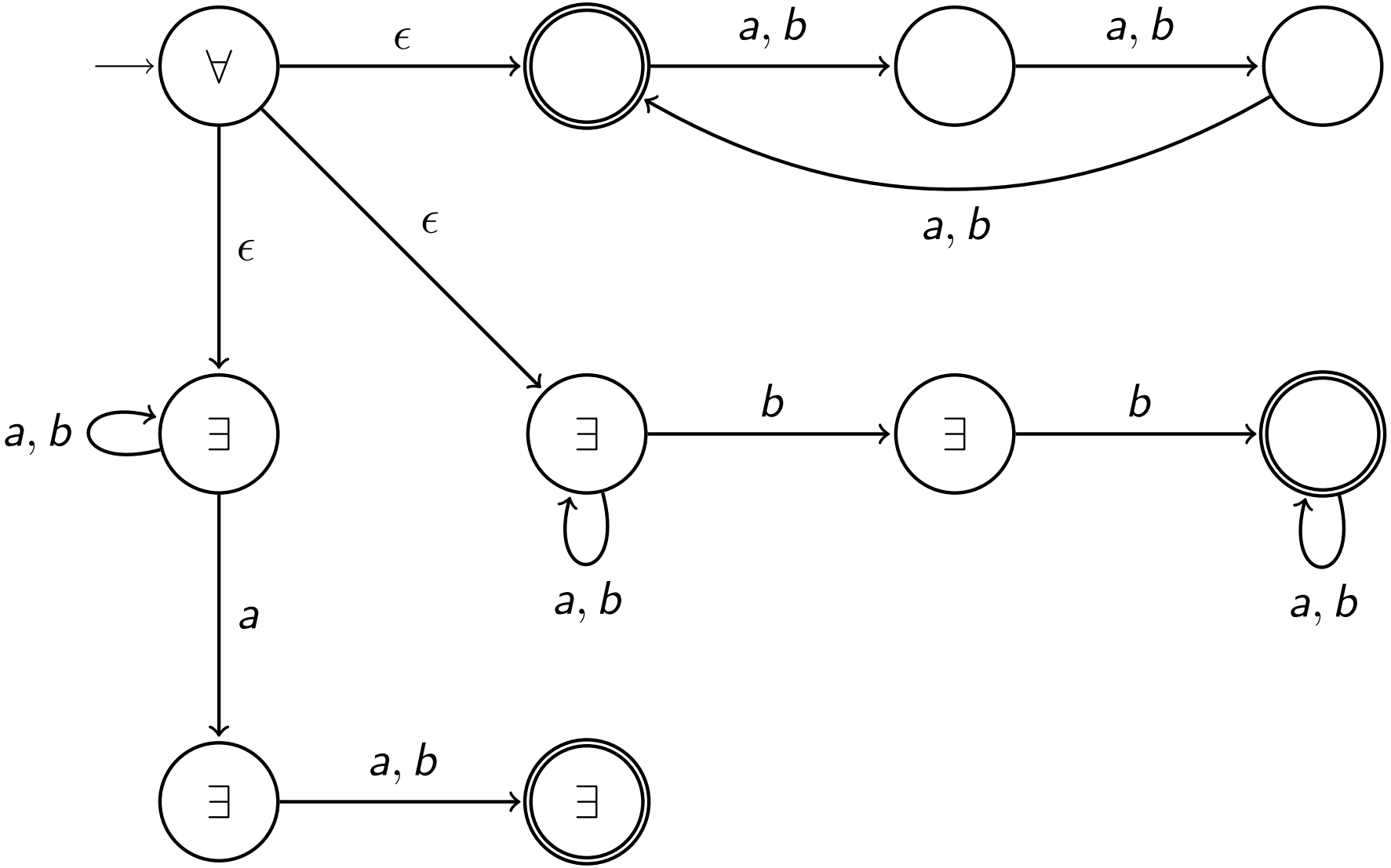
AFA 2

What does this automaton do?



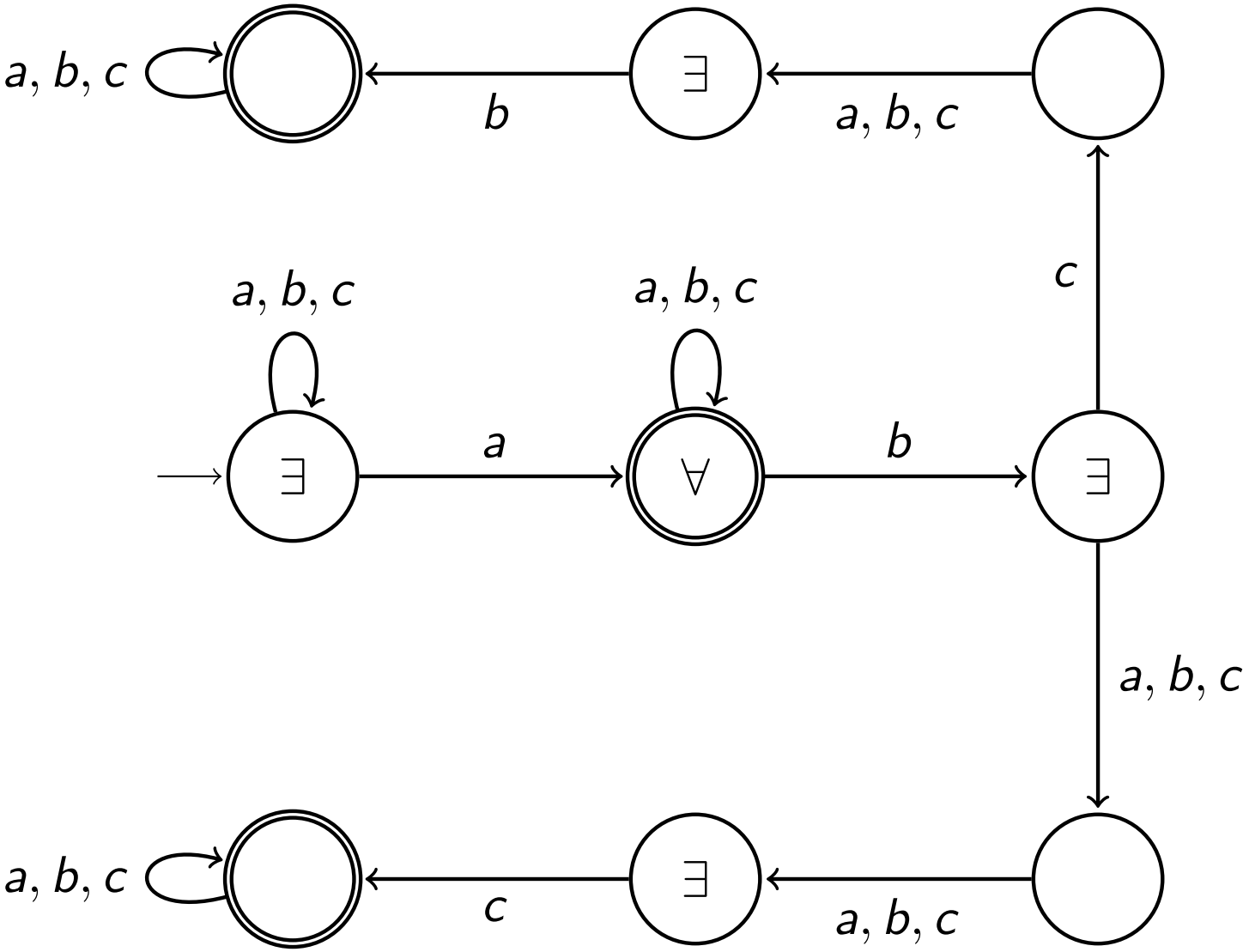
AFA 2

Length is a multiple of 3, and 2nd last letter is a, and contains bb



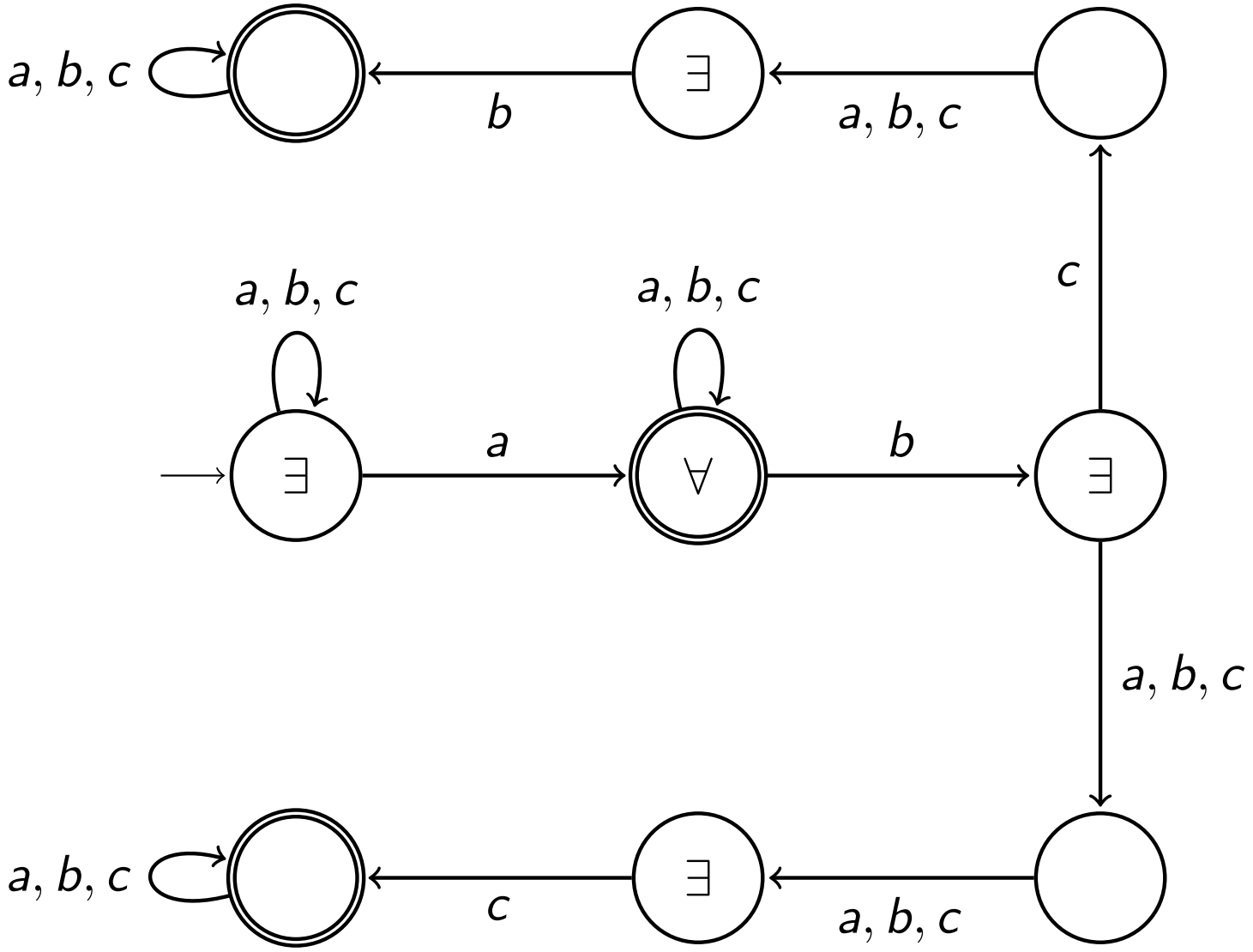
AFA 3

What does this automaton do?



AFA 3

The string contains an a such that, after it, every time b occurs, it is followed by either $c?b$ or $??c$



eg.

abbcbaabccbccc is accepted, *bacabcc* is rejected

Why?

- These automata seem complex... what's the point?
- Very convenient representation for many languages
  - Don't believe me? Try designing a DFA for the previous language!
- Many nice properties
- More 'symmetric' than NFAs
- Close relation to alternating Turing machines

Expressiveness

- Are AFAs more powerful than NFAs? ie. Can they recognise languages that NFAs can't?
- Recall that NFAs are no more powerful than DFAs, due to the subset construction
- Does a similar idea apply to AFAs?
- *drumroll*...

Subset construction

For every AFA, there is an equivalent NFA with 2^n states.

- Proof sketch: Apply a modified subset construction.
- What should the simulating NFA do in an existential state? Easy - just pick one using the NFA's nondeterminism
- What should the simulating NFA do in a universal state? Remember all subsequent states, and pack them into the state
- So new states are sets $Q' \subset 2^Q$
- Which states are final? The states where *every* element is in F
- Corollary: For every AFA, there is an equivalent DFA with 2^{2^n} states (by combining the conversions)

Closure properties

For any two AFAs A, B, there is an AFA for $L(A) \cup L(B)$ with $|A| + |B| + 1$ states.

Proof: Same as for NFAs: add a new $\exists$ initial state with epsilon transitions to the two old initial states.

For any two AFAs A, B, there is an AFA for $L(A) \cap L(B)$ with $|A| + |B| + 1$ states.

Proof: The opposite! Add a new $\forall$ initial state with epsilon transitions to the two old initial states.

- AFAs sure are convenient!

Closure properties

What about complement?

For any AFA A, there is an AFA for $\Sigma^* \setminus L(A)$ with $|A|$ states.

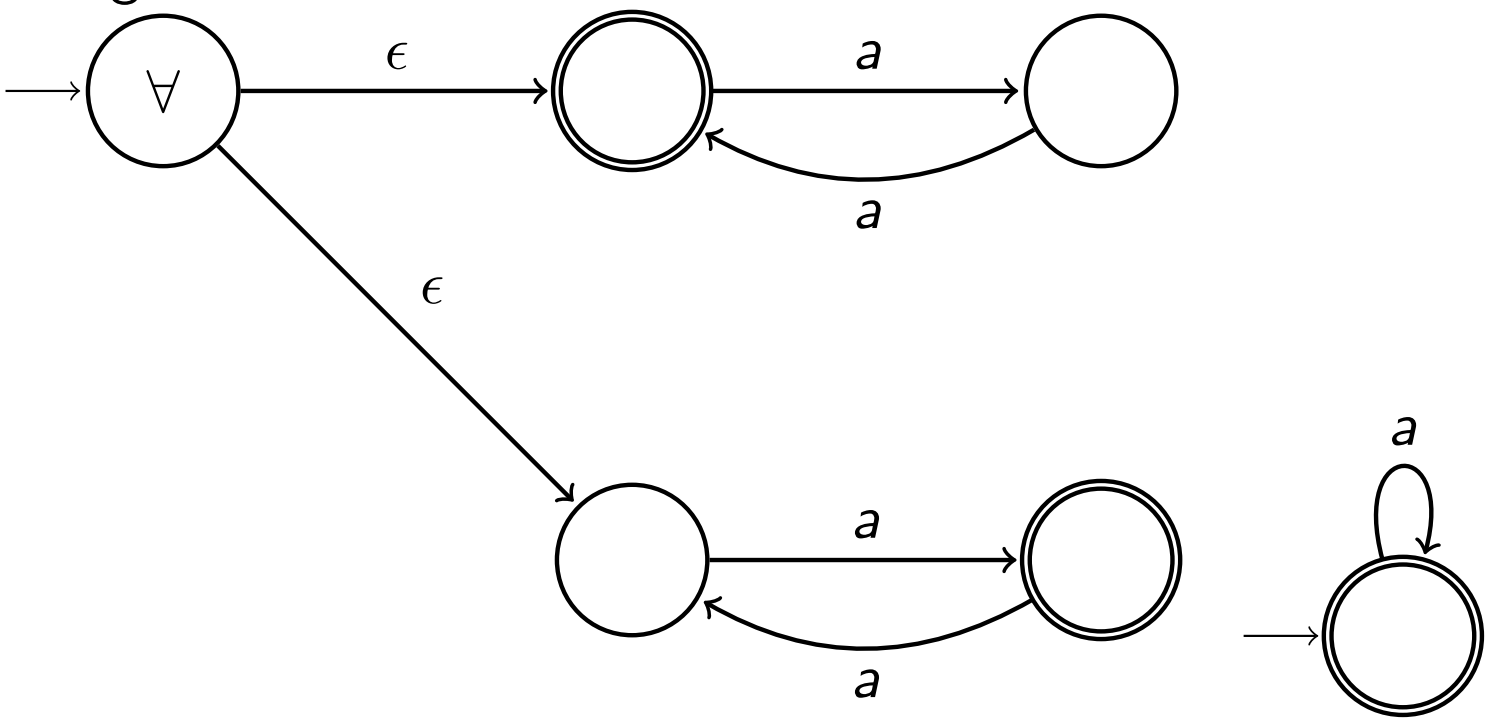
- Idea: Use the fact that $\neg\exists x P \equiv \forall x \neg P$ and $\neg\forall x P \equiv \exists x \neg P$
- Switch $\exists$ and $\forall$ states, and switch final and non-final states
- Why does this work? Induct on the length of the string

A trap

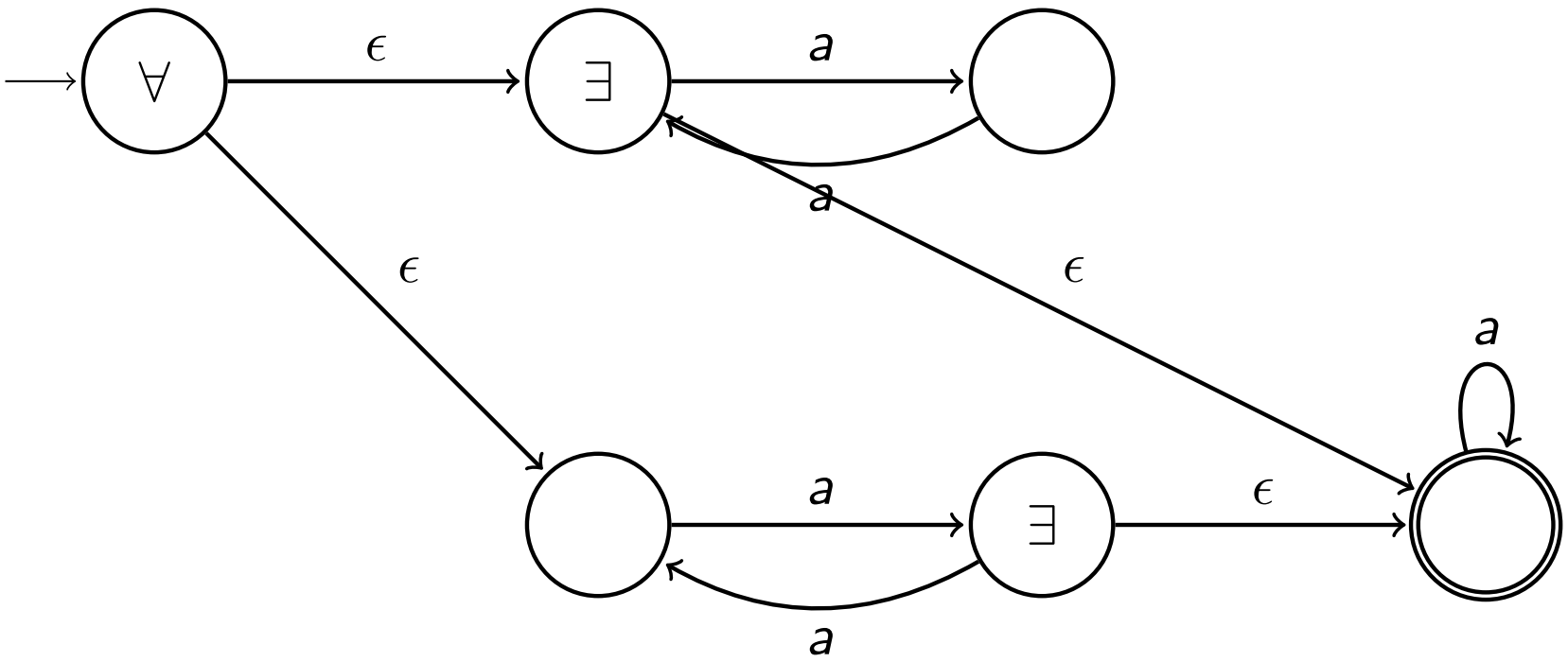
- One common pitfall with AFAs to avoid
- Can we concatenate AFAs efficiently?
- Use the NFA construction: "Give each final state of the first AFA an existential transition to the second AFA's initial state"
- Does this work?

A trap

No! eg:



A trap



A trap

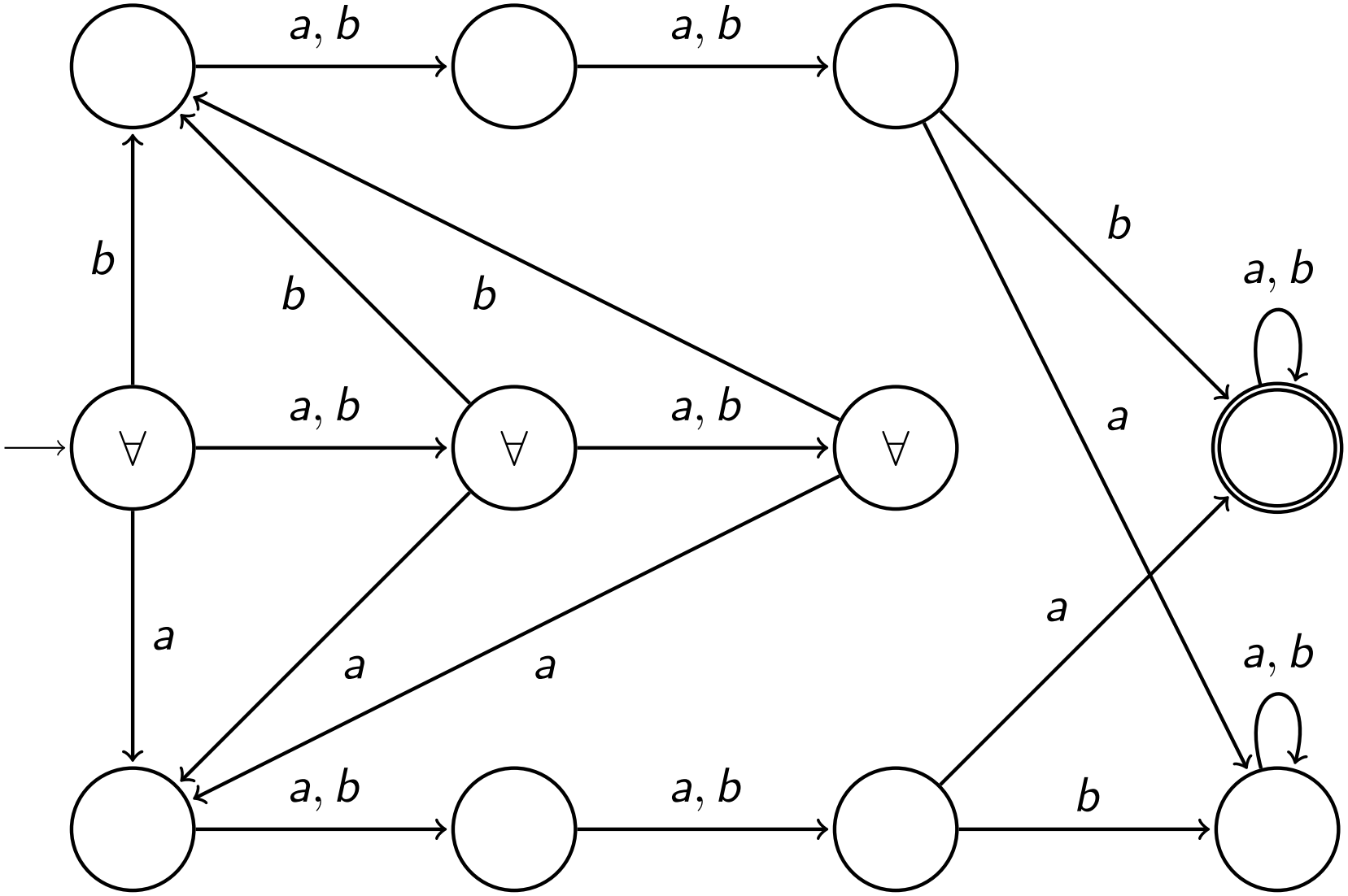
- What went wrong?
- Once two branches separate at a $\forall$ state, they are independent
- So they can take the epsilon transition at different times and break the construction
- Can this be fixed? Can we concatenate two AFAs into an AFA with polynomially many states? (tutorial)

One final use of AFAs

- AFAs are good at bounded string matching
- Check that a string of fixed length n matches another one
- Why? Because universal states = check first letter matches, second letter matches, etc.
- Remember they can't solve most unbounded string matching problems (since such languages are usually non-regular)

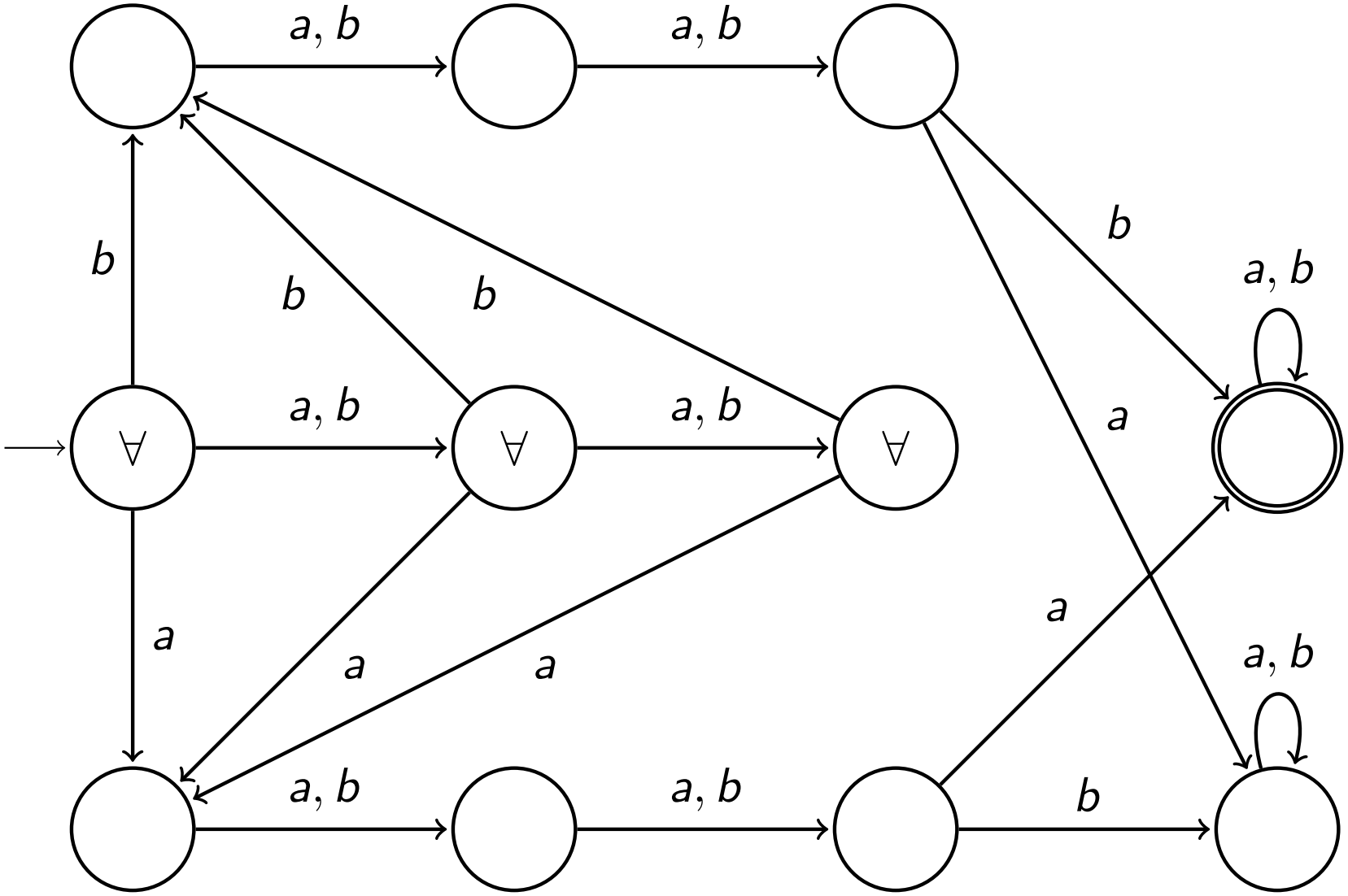
One final example

What bounded string matching problem does this automaton solve?



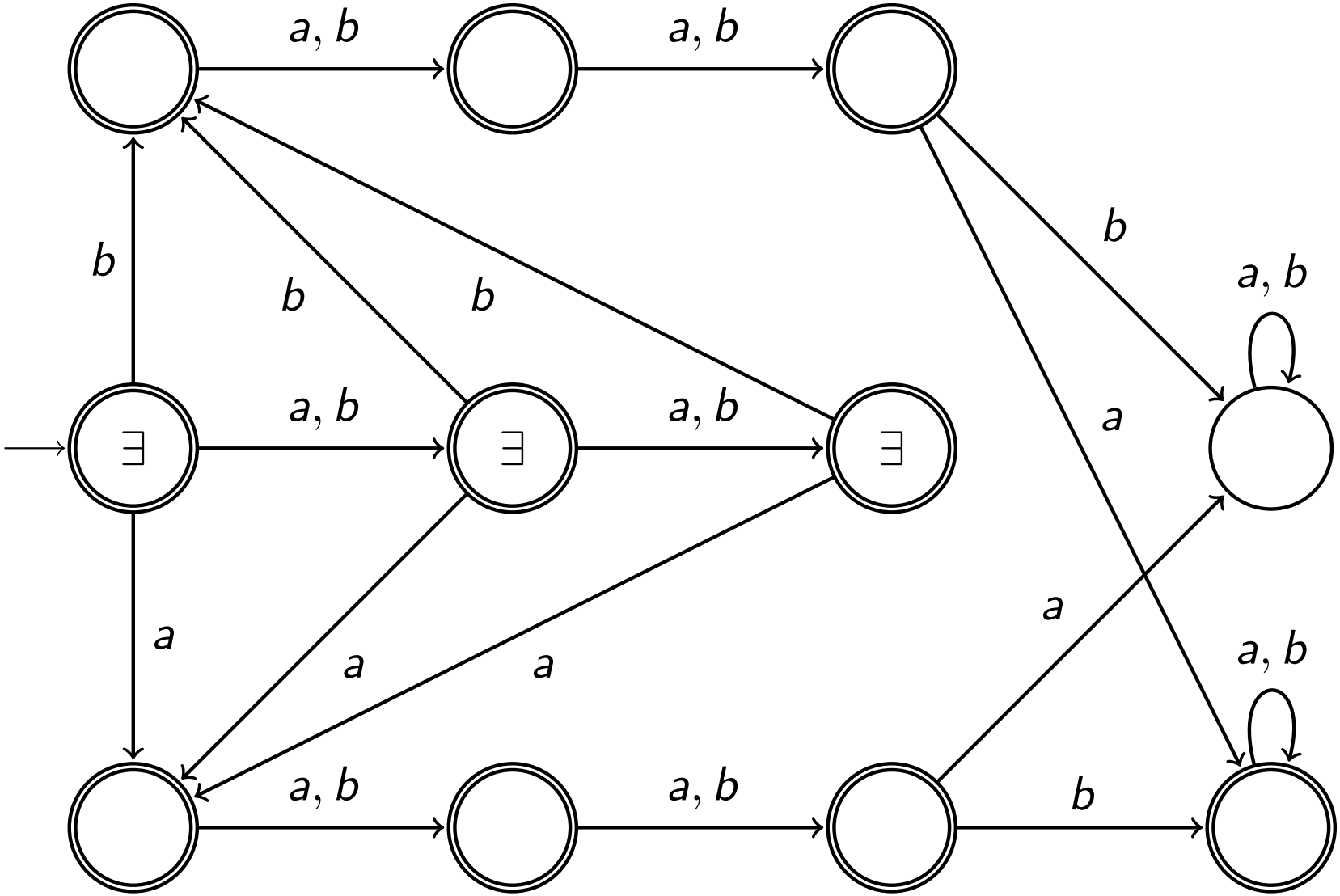
One final example

The string formed by the first 3 letters is the same as the next 3 letters



An alternate view: the complement language

The string formed by the first 3 letters differs from the next 3 letters (or, length < 6)



Formal semantics (not assessable)

We say a state q accepts a string x if:

- $x = \epsilon$ and $q \in F$
- $T(q) = \exists$ and there exists $a \in \Sigma \cup \epsilon$, $y \in \Sigma^*$, and state r such that $x = ay$, $r \in \delta(q, a)$ and r accepts x
- $T(q) = \forall$ and for all $a \in \Sigma \cup \epsilon$, $y \in \Sigma^*$, and state r such that $x = ay$ and $r \in \delta(q, a)$, we have that r accepts x. Furthermore there must exist at least one such combination.

Note that because epsilon cycles are disallowed, a cycle where whether q_1 accepts x depends on whether q_2 accepts x, which in turn depends on whether q_1 accepts x, is not possible.

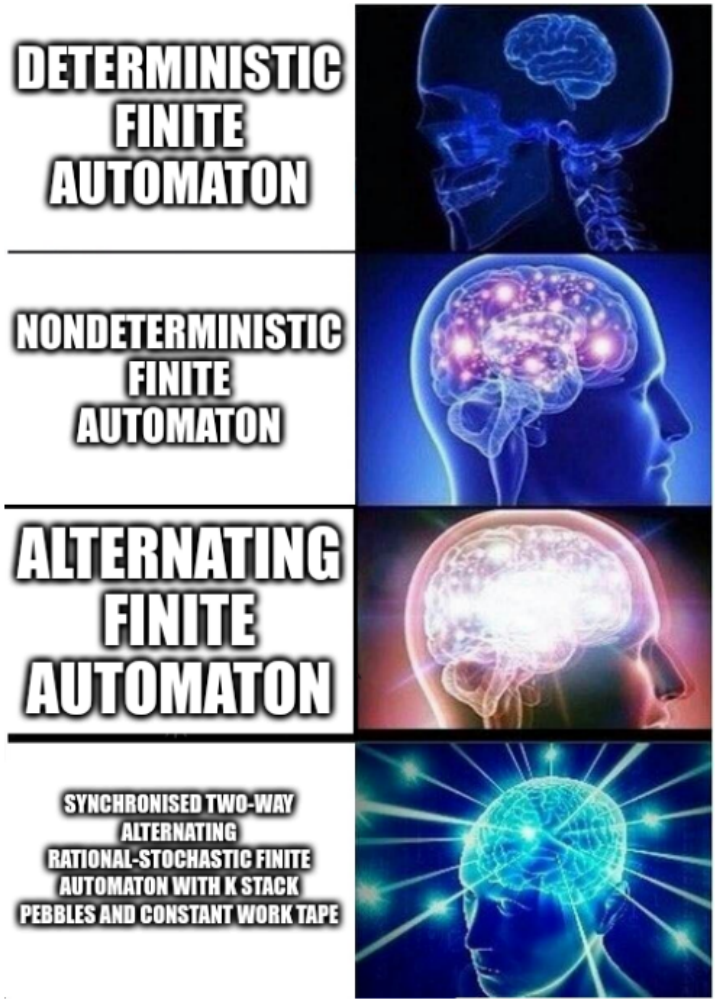
Game semantics (not assessable)

- An alternate way to look at AFA semantics
- Imagine AFA runs as a game between E, who wants to make the AFA accept, and A, who wants to make the AFA reject
- At a $\exists$ state, E chooses a transition
- At a $\forall$ state, A chooses a transition
- Once the AFA has read the entire string, E wins if the AFA is in a final state, otherwise A wins
- Then we interpret the acceptance condition as: E has a winning strategy
- Note: Both players can see the whole string from the start, and plan their moves accordingly

Alternate AFA models (not assessable)

- DFAs and NFAs are relatively standardised
- AFAs have many different versions
- Sometimes epsilon transitions are disallowed
- Sometimes epsilon cycles are allowed
- Sometimes, instead of $\exists\forall$ states, transitions are given by Boolean formulas
- eg. we can transition to q_1 and q_3, or q_4 and q_5
- Sometimes these formulas allow negation, sometimes not
- These differences are sometimes minor, and sometimes change what can be expressed with a small AFA

Summary


DETERMINISTIC FINITE AUTOMATON
NONDETERMINISTIC FINITE AUTOMATON
ALTERNATING FINITE AUTOMATON
SYNCHRONISED TWO-WAY ALTERNATING RATIONAL-STOCHASTIC FINITE AUTOMATON WITH K STACK PEBBLES AND CONSTANT WORK TAPE

Other summary

- AFAs extend NFAs in a similar way to how NFAs extend DFAs
- They recognise the same class of languages (regular languages)
- But they can be exponentially more succinct
- We can do various cool things with AFAs
- Next week: Greg 2: Electric Boogaloo, featuring computability